

Frontend Documentation

Full product + engineering reference — screens, auth UI, navigation, features, styling, deploy.

Stack: **React 19** · **Vite 8** · **TypeScript** · **React Router 7** · **GSAP** · **Recharts** · **Lucide** · **KaTeX**

Live app: <https://myexamprep-theta.vercel.app> · Generated for walkthrough / interview / handover

1. What this frontend is

ExamPrep is a single-page app for competitive exam preparation. After login, users pick a target exam and get a calm LMS shell: daily focus, syllabus notes, revision deck, drills, timed topic tests, full mocks, performance/analytics, MCQ battle, reminders, and an admin console.

- **Repo path:** `frontend/`
- **Brand:** ExamPrep (`src/shared/brand.ts`)
- **Dev:** `npm run dev` (Vite :5173, proxies `/api` → localhost:5000)
- **Build:** `npm run build` → `dist/` on Vercel

2. Table of contents

1. Boot architecture
2. Navigation model (activeView + URLs)
3. Auth UI
4. Every feature / screen
5. Shared services & state
6. Styling & mobile
7. Env + Vercel
8. User journeys to demo
9. 2-minute pitch

3. Boot architecture

```
index.html
├─ main.tsx
│   ├─ ThemeProvider + ExamProvider + ErrorBoundary + HelmetProvider
│   └─ App.tsx (BrowserRouter)
│       └─ Dashboard.tsx ← ONE shell for the whole product
```

There is no multi-page React Router tree of pages. URLs map to an `activeView` string; `Dashboard` decides what to render. That keeps deep-links and reload working while the UI stays one app.

5. Auth UI

Implemented in `features/auth/components/AuthPanel.tsx` + `shared/utils/gsi.ts`. Handlers live in `useStudy`.

Mode	What user does	API
login	Email/username + password	POST /auth/login
register → step-2	Email then password	POST /auth/register
verify	6-digit OTP	POST /auth/otp/verify (+ request)
forgot / reset	Email → OTP → new password	/password/forgot + /reset
Google	Continue with Google	POST /auth/google

Google sign-in (worth explaining)

- Client ID from `VITE_GOOGLE_CLIENT_ID` or runtime `GET /auth/google-config`
- **Credential / ID token first** (FedCM / One Tap) — no client secret needed
- Auth-code popup only if server reports `codeFlowEnabled`
- Mobile prefers official GIS button (popups unreliable)
- Vercel + Vite headers: COOP + Permissions-Policy for identity-credentials

Session: `localStorage` keys `ssc_token`, `ssc_user`. 401 on protected calls clears session.

Auth page is **viewport-locked** (no page scrollbar) — form densifies to fit.

6. Features (what you built for users)

6.1 What to Study (Home)

- Target exam + exam date (ExamPicker / ExamContext)
- Daily focus checklist, subject shortcuts, strong/weak topics from progress
- Wrong-vocab CTA into drills; links to reminders & performance

6.2 Syllabus & Notes

- Official (global) vs My Notes (personal) toggle
- Subject → topics → rich HTML notes (TOC, search, bookmarks, diagrams)
- Start topic test → TCS-style ExamPortal → Results
- Sticky notes + floating notes dock

6.3 Revision Deck

- Multiplication / square / cube tables
- Fraction ↔ percentage reference
- Vocab CRUD + bulk JSON (idioms, word power, OWS)

6.4 Syllabus Roadmap

- Static per-exam syllabus checklist

- Topic status rings (pending / in progress / done), notes modal
- Progress stored client-side

6.5 Daily Drills

Types: table, fraction, percentage, square, cube, vocab, GK / English / Maths / Reasoning MCQ.

- Wrong-log re-attempt
- Related questions
- AI explanation on mistakes
- Add-questions panel for bank uploads

6.6 Topic Exam + Results

- Fullscreen portal: timer, palette, mark-for-review, clear, save & next
- Mobile swipe between questions
- Submit → results with score / accuracy / review

6.7 Full Mocks

- List / create / delete papers (admin create)
- Section limits by exam profile
- Timed multi-section FullMockPortal
- Progress saved via `POST /auth/mock-progress`

6.8 Performance & Analytics

- Performance: attempt tables, status rings, CSV export
- Analytics: Recharts — mock scores, topic distribution, time spent (exam-filtered)

6.9 MCQ Battle (Competition)

- 20 questions × 45s; subjects Mixed / GK / English / Maths / Reasoning
- Submit score → personal best + rank + leaderboard

6.10 Reminders

- Once / daily / weekdays schedules
- Browser notifications + server notification poll while logged in

6.11 Admin

Visible when `user.role === 'admin'` (set by backend ADMIN_EMAIL).

- Map subjects to exams
- TCS bank upload (single / bulk / PDF text extract)
- Mock section patterns
- Institute stats + CSV tools

6.12 ExamPicker

First-login modal: SSC, Banking, Railways, UPSC, CAT, State PSC, Other — drives marking scheme, mock shape, and Today Focus copy via `examProfiles.ts`.

7. Shared layer

Piece	Role
<code>apiService</code>	HTTP client, Bearer token, timeouts, 401 session clear
<code>ThemeContext</code>	Light / dark (<code>data-theme</code>)
<code>ExamContext</code>	Target exam, date, subjects from <code>/exam-config</code>
<code>useStudy</code>	Auth + syllabus + tests + progress (central hook)
<code>useDrills</code> / <code>useMockTests</code>	Feature data hooks
<code>useAppNavigation</code>	<code>activeView</code> ↔ URL
<code>gsi.ts</code>	Google Identity Services helpers
<code>backTrap</code>	Smart browser-back inside the SPA
<code>notesMarkup</code> / <code>KaTeX</code>	Safe / pretty notes & MCQ text
<code>appToast</code>	Themed toast notifications

8. Styling & mobile

- Fonts: **Manrope** (UI) + **Outfit** (display)
- Design tokens in `styles/index.css` — CSS variables for light/dark
- Global layers: premium-ui, layout, workspace, modals, mobile-layout, mobile-sleek, design-overhaul
- Feature CSS co-located (auth, exam, drills, study, etc.)
- GSAP for auth entrance + workspace transitions (respects reduced motion)
- Mobile: hamburger drawer sidebar; exams fullscreen; touch swipe; auth no-scroll viewport

9. Environment & deploy

Variable	Use
<code>VITE_API_URL</code>	Optional local API base (ignored on Vercel prod)
<code>VITE_GOOGLE_CLIENT_ID</code>	Google Web client (also injectable from backend <code>.env</code> at build)

API base rules: on Vercel production → same-origin `/api` (rewrite to Render). Locally → proxy or `localhost:5000`.

vercel.json

- Rewrite `/api/:path*` → `https://ssc-0p96.onrender.com/api/:path*`

- SPA fallback → `index.html`
- Headers: COOP, nosniff, referrer, X-Frame-Options DENY, Permissions-Policy, CSP

10. Folder map

```
frontend/
├─ index.html
├─ vercel.json
├─ vite.config.ts
├─ src/
│   ├── main.tsx
│   ├── app/                # App + path helpers
│   ├── types/app.ts
│   ├── features/
│   │   ├── dashboard/     # shell
│   │   ├── auth/
│   │   ├── home/
│   │   ├── study/
│   │   ├── drills/
│   │   ├── exam/
│   │   ├── mock-tests/
│   │   ├── roadmap/
│   │   ├── analytics/
│   │   ├── competition/
│   │   ├── reminders/
│   │   └─ admin/
│   ├── shared/            # api, context, hooks, utils, UI
│   └─ styles/             # tokens + layout + mobile
```

11. Demo journeys (use these when explaining)

1. **Sign up:** Register → OTP → ExamPicker → Home
2. **Study:** Syllabus → notes → Start test → Results
3. **Speed:** Daily Drills → wrong-log → AI explain
4. **Paper:** Full Mocks → timed portal → Performance / Analytics
5. **Compete:** MCQ Battle → leaderboard
6. **Admin:** Map subjects / upload TCS bank / mock pattern

12. How to explain this frontend (2-minute pitch)

1. "I built ExamPrep as a React + Vite SPA — one Dashboard shell, URL-synced views, lazy-loaded workspaces."
2. "Auth UI covers password, OTP, forgot-reset, and Google GIS with credential-first + FedCM headers."
3. "Product surface includes syllabus notes, revision, adaptive drills, TCS-style exams, full mocks, analytics, battle mode, and reminders."
4. "Exam-aware UX: ExamPicker + profiles drive marking, mock sections, and Today Focus."
5. "Shipped on Vercel with /api rewrite to Render, CSP/COOP headers, and a polished light/dark mobile-ready UI."

